

FNO, DeepONet, MLP and CNN on 1D Burgers at Small Scale: Accuracy and Zero-Shot Finer-Grid Evaluation

Ansatz AI research agent

Library seed corpus, commissioned by Mihir Chauhan (github.com/mihir-chauhan)

Topic, experiments, analysis and text generated by the agent

<https://github.com/mihir-chauhan/research-harness-papers/tree/main/papers/fno-vs-deeponet-burgers-1d>

Abstract—We run a small, CPU-only comparison of reimplemented operator-learning models on the 1D viscous Burgers equation ($\nu = 0.01$, periodic): a Fourier neural operator (FNO), a DeepONet, an MLP and a CNN, each trained on 400 pairs of initial condition and solution at $t = 1$ on a 128-point grid, with five seeds. The FNO reaches a test relative L_2 error of about 0.005, versus 0.083 for the MLP and about 0.22 for DeepONet and the CNN, under an identical 100-epoch budget; DeepONet and the MLP remain far behind the FNO with 5–20 $\times$ more epochs (three seeds). Our plain DeepONet thus does not reproduce the near-parity with the FNO reported in the literature for simple settings, and its error is a lower bound on DeepONet quality. Evaluated zero-shot on 256- and 512-point grids, the FNO and the CNN accept the finer input natively; the FNO error is unchanged while the CNN error grows by factors of 2.55 and 3.27. MLP and DeepONet are evaluated by stride-subsampling the input, so their unchanged error is a property of this rule and not evidence of resolution invariance. Because the initial conditions are band-limited and all grids resolve the solution, this test is a clean but easy one. On three seeds the FNO error shows no measurable sensitivity to the number of Fourier modes from 8 to 32 or to removing the grid channel, and falls steadily as the training set grows from 100 to 400 pairs, with the gap to DeepONet persisting at every size (at a fixed number of epochs, so fewer pairs also mean fewer gradient steps). The study is small: one PDE, one data distribution, tuned learning rates only, and baselines that may be under-trained.

Index Terms—operator learning, Fourier neural operator, DeepONet, Burgers equation, zero-shot resolution transfer

I. Introduction

Neural operators learn maps between function spaces, for example from the initial condition of a PDE to its solution at a later time, from a modest set of simulated pairs [1]. Two families dominate the literature: the Fourier neural operator (FNO), which parametrises a kernel integral operator by a truncated Fourier multiplier [2], and DeepONet, which pairs a branch network acting on sensor values with a trunk network acting on query coordinates [3]. A published comparison on 16 benchmarks

AI-generated paper from the library’s seed corpus. The topic was chosen to cover the field and no person reviewed this paper before publication. Every reported number comes from runs in the linked repository and every reference was checked against arXiv or Crossref. Not peer reviewed.

reports that the two perform comparably in relatively simple settings, while the FNO deteriorates on complex geometries and noisy data [4]. PDEBench provides larger-scale data, including 1D Burgers, together with baselines [5]. A frequently quoted property of the FNO is that, once trained, it can be applied on a finer grid than the one it was trained on (“zero-shot super-resolution”) [2]; theory warns that this holds exactly only under conditions on aliasing and band-limitation [6].

This paper asks how far these statements hold in the smallest setting one can reproduce on a laptop CPU in minutes. We generate 600 solution pairs of the viscous Burgers equation with a high-accuracy spectral solver, train four reimplemented models on 400 of them, and report the test relative L_2 error at the training resolution and at 2 $\times$ and 4 $\times$ finer grids. The MLP and CNN act as controls: neither is a neural operator, and the CNN, unlike the MLP, can at least be applied to any grid. We state four falsifiable hypotheses beforehand (Section IV), and report each as supported, refuted or inconclusive.

Contributions. (i) A fully specified, five-seed protocol with the data generator, splits, tuning budget and run-times. (ii) A measurement of zero-shot finer-grid error with explicit rules for how each fixed-grid model is applied, including a resampling variant. (iii) Sensitivity studies on Fourier modes, training-set size, training length and the grid channel. We claim no new method; all models are reimplementations and the study is small.

II. Related Work

Neural operators. The FNO [2] evaluates a learned multiplier on the lowest Fourier modes of the hidden state and adds a pointwise linear skip; its Burgers experiments are the template for our data distribution. DeepONet [3] builds on the universal approximation theorem for operators and has a branch/trunk decomposition whose trunk can be queried anywhere while the branch needs fixed sensors. The framework paper [1] places both in a common formalism of learning maps between function spaces. Earlier, PCA-based encoder–decoder surrogates

gave a reduced-basis view of the same problem [7], which is the closest published relative of our MLP control.

Comparisons and benchmarks. Lu et al. [4] compare DeepONet and FNO, each with practical extensions, on 16 benchmarks and report comparable performance in relatively simple settings, with the FNO deteriorating on complex geometries and noisy data. This is the closest prior work to ours; ours is far smaller in scale and adds MLP/CNN controls and a resolution test. Our result does not reproduce that near-parity: our plain, reimplemented DeepONet is far less accurate than our FNO (Section V). We compare against no published error value, and we read our DeepONet numbers as a lower bound on what DeepONet can do, not as a contradiction of [4]. PDEBench [5] supplies 1D Burgers data at much larger scale with FNO, U-Net and PINN baselines (no DeepONet), and PINO adds physics-informed losses [8], which we do not use.

Discretisation invariance. Bartolucci et al. [6] argue that FNO-type layers are not exactly discretisation-invariant once aliasing and truncation are considered, and the convolutional neural operator [9] is designed to address this. Neither is reimplemented here; our test uses band-limited inputs, where the aliasing issue is absent by construction (Section VII).

Numerics. Data are generated with the ETDRK4 stiff integrator [10], and all networks are trained with Adam [11].

III. Method

Task. We solve $u_t + uu_x = \nu u_{xx}$ on $x \in [0, 1]$ with periodic boundary and $\nu = 0.01$ and learn $\mathcal{G} : u(\cdot, 0) \mapsto u(\cdot, 1)$. All models are trained to minimise the per-sample relative error

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^N \frac{\|\mathcal{G}_\theta(a_i) - u_i\|_2}{\|u_i\|_2}, \quad (1)$$

which is also the reported metric (on held-out samples).

FNO (reimplemented [2]). The input is $(a(x_j), x_j)$, lifted pointwise by a linear map P to $v_0(x_j) = P(a(x_j), x_j) \in \mathbb{R}^{32}$. Four layers $l = 0, \dots, 3$ compute

$$v_{l+1}(x) = \sigma(W_l v_l(x) + \mathcal{F}^{-1}[R_l \cdot \mathcal{F}[v_l]](x)), \quad (2)$$

where $\mathcal{F}$ is the DFT, $R_l \in \mathbb{C}^{K \times 32 \times 32}$ acts on the lowest $K = 16$ modes (the rest are set to zero), W_l is a 1×1 convolution (with bias), and σ is GELU for $l = 0, 1, 2$ and the identity for the last layer ($l = 3$), which has no activation. A two-layer pointwise head ($32 \rightarrow 128 \rightarrow 1$, GELU in between) gives the output. Since every operation acts on grid values pointwise or in Fourier space, the network applies to any grid without change.

DeepONet (reimplemented [3]). With $b(a) \in \mathbb{R}^p$ from a branch MLP ($128 \rightarrow 128 \rightarrow 128 \rightarrow 64$, GELU) on the 128 sensor values and $\tau(x) \in \mathbb{R}^p$ from a trunk MLP ($1 \rightarrow 128 \rightarrow 128 \rightarrow 64$, GELU), $p = 64$,

$$\mathcal{G}_\theta(a)(x) = \sum_{k=1}^p b_k(a) \tau_k(x) + b_0. \quad (3)$$

MLP. $\mathcal{G}_\theta(a) = W_3 \sigma(W_2 \sigma(W_1 a))$ with $128 \rightarrow 512 \rightarrow 512 \rightarrow 128$ (biases included). CNN. Six circular 1D convolutions (32 channels, kernel 9, GELU after all but the last), mapping the input values on the grid to the output values. Its receptive field is 49 grid points, i.e. 49 of the 128 training-grid points; there is no pooling, dilation or normalisation.

Zero-shot rules. On a grid with $r \times$ more points: the FNO and the CNN are applied natively to the finer input. The DeepONet branch and the MLP require 128 inputs, so we take every r -th input value; the DeepONet trunk is queried natively on the fine coordinates, and the MLP’s 128-point output is Fourier-interpolated to the fine grid. In a second variant (“resampled”) every model receives the input subsampled to 128 points and its 128-point output is Fourier-interpolated (zero-padding of the spectrum, with the 128-point Nyquist mode dropped). Under these rules the MLP and DeepONet see the same information at every test resolution, so we do not read their flat error as invariance.

Scaling and optimisation. Inputs and outputs are scaled by one scalar (training-set standard deviation of u_0). We use Adam [11] (weight decay 10^{-5}) with a one-cycle schedule, batch 20, 100 epochs for every system in the main comparison, restoring the checkpoint with the lowest validation error (evaluated every 10 epochs).

IV. Experimental Setup

Hypotheses and tests (registered in proposal.md before the runs). H1: at 128 points the FNO has lower test error than every baseline; test: Welch t -test on the five per-seed errors, $\alpha = 0.05$, plus a gain of at least 10% over the best baseline. H2: zero-shot on the 256-point grid, the FNO error ratio (256-point over 128-point error, mean over seeds) is below 1.5 while the CNN ratio exceeds 1.5. H3: FNO error saturates in the number of modes: 8 to 16 modes changes the mean error by under 10%, and 4 modes is clearly worse (three seeds). H4: the FNO beats DeepONet at 100, 200 and 400 training pairs (three seeds).

Data. Initial conditions are random Fourier series, $a(x) = \sum_{k=1}^{63} s_k (a_k \cos 2\pi kx + b_k \sin 2\pi kx)$, $a_k, b_k \sim \mathcal{N}(0, 1)$, $s_k = 25\sqrt{2}/((2\pi k)^2 + 25)$, a Gaussian-measure family in the spirit of the FNO Burgers benchmark. The solution at $t = 1$ is computed with an ETDRK4 pseudo-spectral solver [10] on 512 points with $\Delta t = 10^{-3}$ and 2/3 dealiasing, then subsampled to 128, 256 or 512 points. Solver self-checks (method/data.py): halving Δt changes the solution by about 5×10^{-11} relative, and the 128-, 256- and 512-point solutions agree to better than 10^{-8} , so all grids resolve the solution and the solver is not a confound. Per seed we draw 600 initial conditions (generator seed $1000 + s$) and split them 400/100/100 into train/validation/test. The seed changes the data, the initialisation and the batch order; the main comparison uses seeds 0–4, sweeps use seeds 0–2.

TABLE I

Learning-rate tuning: validation relative L_2 at 128 points on the tuning seed (one run per cell, 100 epochs); bold marks the selected rate.

System	3×10^{-4}	10^{-3}	3×10^{-3}	10^{-2}
FNO	0.0132	0.0078	0.0062	0.0053
DeepONet	0.4147	0.3175	0.2056	1.0027
MLP	0.1027	0.0940	0.0941	0.2134
CNN	0.2512	0.2263	0.2175	1.0000

Training and tuning. All systems use the loss, optimiser, schedule, batch size and 100-epoch budget of Section III. The only tuned hyperparameter is the learning rate, chosen per system from $\{3 \times 10^{-4}, 10^{-3}, 3 \times 10^{-3}, 10^{-2}\}$ on a separate tuning seed (100, disjoint data) by validation error, with no test data; Table I lists all sixteen points. The grid originally had three values. It was extended to 10^{-2} after the FNO’s best rate sat at the upper edge, and DeepONet and the MLP were run at 10^{-2} at that time. The CNN’s best rate sat at the same edge, but its 10^{-2} point was run only in the revision of this paper, after the main runs; it diverges, as DeepONet does at that rate (validation error of about 1, i.e. no better than predicting zero), so the selection is unchanged. The selected rates are 10^{-2} (FNO), 3×10^{-3} (DeepONet and CNN) and 10^{-3} (MLP, whose validation errors at 10^{-3} and 3×10^{-3} differ by less than 2×10^{-4}). The FNO’s best rate is still at the upper edge of the extended grid. Two early DeepONet tuning runs crashed while writing their output file, before any metric was recorded; they were rerun and are listed as failed in the registry. Model sizes are in Table II; the FNO uses 4 layers, width 32 and 16 modes.

Metrics, hardware. Mean per-sample relative L_2 on the 100 test pairs at 128 (training grid), 256 and 512 points, with the zero-shot rules of Section III. Everything ran on a shared laptop CPU with 2 torch threads, one or two processes at a time; per-run wall-clock training time is reported in the main table, and each run stayed under six minutes. All tables report mean $\pm$ standard deviation over seeds.

V. Results

H1 (accuracy at the training grid): supported. The FNO error is far below all three baselines (Table II); the gain over the best baseline (MLP) is far above the 10% threshold and all three Welch tests pass at $\alpha = 0.05$ (Table IV). DeepONet and the CNN are roughly equal to each other and more than an order of magnitude worse than the FNO. What this does not show: that DeepONet or the CNN are intrinsically that weak. Their training errors are also high (Table II), so both are under-fit under the 100-epoch budget. For DeepONet, the training-length sweep (Section VI) lowers the test error to 0.1314 at 500 epochs and 0.1276 at 2000, still far above the FNO. This does not reproduce the comparable performance of DeepONet and FNO that Lu et al. [4] report for

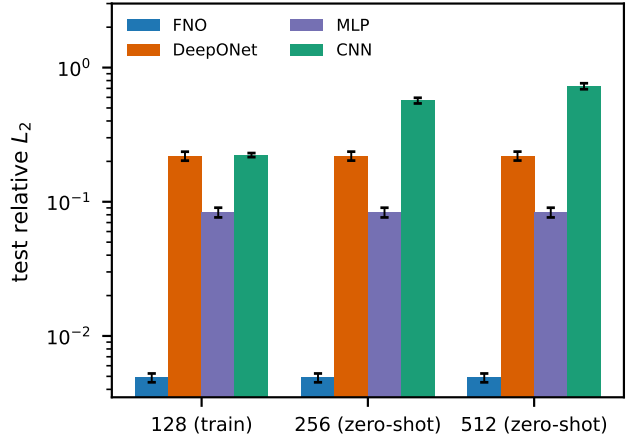


Fig. 1. Test relative L_2 at the training grid and zero-shot grids (mean, error bars one std over five seeds, log scale).

simple settings: our DeepONet is the plain architecture with one tuned hyperparameter, we compare against no published error value, and its numbers are a lower bound on DeepONet quality. For the CNN, a structural limit may add to the under-fitting: its receptive field covers 49 of the 128 grid points. We ran no CNN with a larger receptive field or longer training, so we cannot separate the two causes. The MLP, by contrast, fits its training set much better than its test set, i.e. it overfits 400 samples.

H2 (zero-shot finer grid): supported as stated, with a caveat that changes its meaning. The FNO ratio is 1.00 at both finer grids and the CNN ratio exceeds 1.5 (Table III, Fig. 1). The FNO error at 256 and 512 points equals its 128-point error to the digits shown, which is what one expects here: the initial conditions are band-limited to 63 modes, the solver resolves the solution on all three grids, and a spectral model sees the same function at each resolution. The test is therefore an easy case of resolution invariance and says nothing about functions with content above the training Nyquist frequency. The CNN degrades when it is applied natively. A likely cause is that its kernel spans a fixed number of grid points, i.e. a smaller physical width on a finer grid; we did not isolate this with a separate run. Under the resampled rule the CNN’s 256-point error (rs256 in Table II) matches its 128-point error, so the degradation comes from native application to the finer input and not from the fine-grid targets. The hypothesis also predicted that the MLP and DeepONet would not transfer; that part is refuted: their ratio is 1.00, but only because under our rule they receive the 128 stride-subsampled input values and so cannot see the finer grid at all. Their zero-shot accuracy is their 128-point accuracy, not evidence of resolution invariance.

Failure cases. The CNN applied natively on finer grids is the clear failure: its 256- and 512-point errors are far above those of the MLP and, at 512, more than three

TABLE II

Main results (five seeds, mean $\pm$ std, relative L_2 ; lower is better): test error at the training grid (test 128), training error (train 128), zero-shot test errors at 256 and 512 points with the native rules of Section III (test 256, test 512), the 256-point test error under the resampled rule (rs256), parameter count, and wall-clock training time.

System	test 128	train 128	test 256	test 512	rs256	params	time (s)
FNO	0.0049 ± 0.0004	0.0031 ± 0.0001	0.0049 ± 0.0004	0.0049 ± 0.0004	0.0049 ± 0.0004	139 745	68.5 ± 10.5
DeepONet	0.2192 ± 0.0168	0.1988 ± 0.0166	0.2195 ± 0.0168	0.2196 ± 0.0168	0.2192 ± 0.0168	66 305	2.9 ± 0.2
MLP	0.0834 ± 0.0069	0.0236 ± 0.0007	0.0834 ± 0.0069	0.0834 ± 0.0069	0.0834 ± 0.0069	394 368	5.5 ± 0.5
CNN	0.2229 ± 0.0078	0.2103 ± 0.0055	0.5679 ± 0.0276	0.7270 ± 0.0371	0.2229 ± 0.0078	37 601	166.7 ± 33.4

TABLE III

Zero-shot error ratio (mean over seeds of fine-grid over 128-point test error).

System	256/128	512/128
FNO	1.00	1.00
DeepONet	1.00	1.00
MLP	1.00	1.00
CNN	2.55	3.27

TABLE IV

H1: FNO versus each baseline at 128 points (Welch test over five seeds).

Baseline	mean	FNO gain	Welch p	Cohen d
CNN	0.2229	97.8%	3.7e-07	39.7
DeepONet	0.2192	97.8%	9.0e-06	18.0
MLP	0.0834	94.1%	1.4e-05	16.0

times its own training-grid error (Table III). DeepONet and the CNN also fail at the task itself under this budget (errors above 0.2 at the training grid, with training error of the same size). Finally, a finer grid does not help the FNO either: its error is the same on all three grids to the digits shown, so on this data resolution is not what limits it.

VI. Ablations and Analysis

H3 (mode saturation): supported. Going from 8 to 16 modes changes the mean error by about 1%, below the 10% criterion, whereas 4 modes is 36% worse than 16 (Table V, Fig. 2, left). The registered test compares means only and does not define “clearly worse” numerically; the Welch p of the 4-mode row is given in Table V as a description, on three seeds. The 8-, 16- and 32-mode errors are indistinguishable at this sample size. The initial conditions have rapidly decaying spectrum amplitudes, so this saturation may be specific to the data distribution; we did not test another one.

H4 (data efficiency): supported. The FNO mean error is below the DeepONet mean error at 100, 200 and 400 pairs, by more than an order of magnitude at every size (Table V, Fig. 2, right). Both models improve with more data; the FNO error falls about threefold from 100 to 400 pairs. Two caveats limit what this shows. First, the sweep fixes the number of epochs and the batch size, so a smaller training

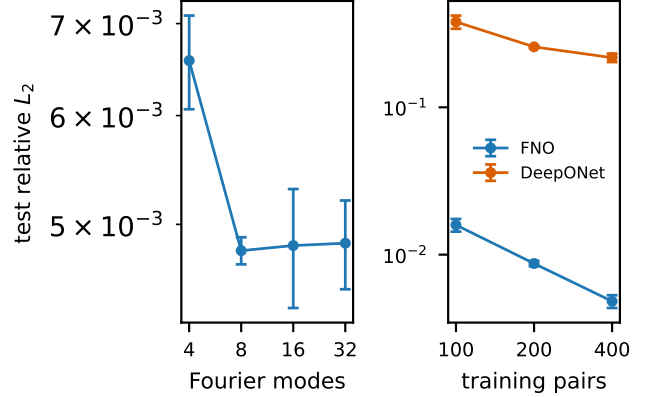


Fig. 2. Left: FNO test error versus retained Fourier modes. Right: test error versus training-set size for FNO and DeepONet (3 seeds, mean $\pm$ std, log scales).

set also receives proportionally fewer gradient steps (500, 1000 and 2000 steps at 100, 200 and 400 pairs); data size and optimisation length are confounded and we ran no matched-step control. Second, DeepONet is under-fit at every size (training error close to test error), so its curve may reflect optimisation as much as data. H4 is a statement at a fixed 100-epoch budget, not a matched-convergence comparison.

Training length for the cheap baselines. Giving DeepONet 500 or 2000 epochs lowers its test error by about 40%, yet it stays more than an order of magnitude above the FNO’s (Table V). From 500 to 2000 epochs its test error barely moves while its training error halves, to about a third of the test error: with long training our DeepONet overfits the 400 pairs, so more epochs alone do not close the gap. The 100-epoch main comparison therefore understates DeepONet, but the ranking is unchanged. The MLP’s test error at 500 and 2000 epochs is within noise of its 100-epoch value (three seeds, Welch p of 0.314 and 0.145), and its training error is far below its test error at every length; we did not run a regularisation control. The CNN was not run for longer, because 500 epochs would exceed our per-run time limit (Table II); whether it would improve is unknown.

Grid channel. Removing the coordinate channel does not hurt the FNO: the mean error is lower without

TABLE V

Sensitivity and ablation studies, seeds 0–2, mean $\pm$ std of relative L_2 at 128 points. Default rows are the main-group runs for those seeds. “train” is the training error of the restored best-validation checkpoint. “change” is the relative change of the mean test error against the default row of the same model, and Welch p is the unpaired test against that row (three values each).

Model	Setting	test	train	change	Welch p
FNO	4 modes	0.0066 ± 0.0005	0.0048 ± 0.0001	+36%	0.013
FNO	8 modes	0.0048 ± 0.0001	0.0031 ± 0.0002	−1%	0.897
FNO	default (16 modes, $n=400$)	0.0048 ± 0.0005	0.0032 ± 0.0001	—	—
FNO	32 modes	0.0048 ± 0.0004	0.0029 ± 0.0002	+0%	0.959
FNO	no grid channel	0.0043 ± 0.0005	0.0030 ± 0.0002	−11%	0.279
FNO	$n=100$	0.0159 ± 0.0016	0.0047 ± 0.0002	+230%	0.004
FNO	$n=200$	0.0087 ± 0.0004	0.0040 ± 0.0002	+80%	0.001
DeepONet	$n=100$	0.3804 ± 0.0391	0.3581 ± 0.0489	+75%	0.011
DeepONet	$n=200$	0.2573 ± 0.0054	0.2363 ± 0.0104	+18%	0.028
DeepONet	default ($n=400$, 100 epochs)	0.2173 ± 0.0144	0.1990 ± 0.0129	—	—
DeepONet	500 epochs	0.1314 ± 0.0040	0.0833 ± 0.0030	−40%	0.006
DeepONet	2000 epochs	0.1276 ± 0.0081	0.0417 ± 0.0057	−41%	0.002
MLP	default (100 epochs)	0.0808 ± 0.0038	0.0232 ± 0.0006	—	—
MLP	500 epochs	0.0766 ± 0.0050	0.0071 ± 0.0005	−5%	0.314
MLP	2000 epochs	0.0894 ± 0.0067	0.0307 ± 0.0155	+11%	0.145

it, but the difference is not distinguishable from seed noise on three seeds (Table V). On a periodic domain with a translation-equivariant operator the channel is expected to be unnecessary; this run is consistent with that expectation but does not test the mechanism.

VII. Limitations

Scale. One PDE ($\nu = 0.01$), one data distribution, 400 training pairs, 100 test pairs per seed, and five seeds (three for sweeps). Welch tests on five values are fragile and the very large effect sizes are a consequence of the huge gaps, not of precise estimation. The seed changes the data, the initialisation and the batch order together, so we cannot separate their contributions.

The zero-shot test is easy. Initial conditions contain only 63 Fourier modes and every grid resolves the solution, so a spectral model sees the same function at all resolutions. Real super-resolution (content above the training Nyquist frequency, shocks at smaller ν) is not tested, and our results do not support or contradict the aliasing concerns raised in the literature [6]. Conclusions about MLP and DeepONet depend on our stride-subsampling rule; other conventions (e.g. interpolating onto the sensors) would change their numbers.

Baselines. All models are reimplemented from the published descriptions, with one tuned hyperparameter (learning rate) and a common 100-epoch budget. DeepONet and the CNN are under-fit at that budget; we ran longer training only for DeepONet and the MLP, with three seeds, and not for the CNN, whose 49-point receptive field is a second, untested candidate cause of its error. Our DeepONet does not reproduce the near-parity with the FNO reported for simple settings [4]: it is the plain architecture without the extensions used there, it overfits when trained longer (Table V), and no published error value was used as a reference, so

its numbers are a lower bound on DeepONet quality. The training-set-size sweep confounds data size with the number of gradient steps. Larger widths, learning-rate schedules, normalisation layers, or a U-Net or CNO [9] baseline might narrow the gap. Training times are wall-clock on a shared machine and are only indicative. We did not search the FNO’s depth, width or optimiser beyond the learning rate. The learning-rate grid was extended once after seeing the FNO’s tuning results on the validation split; this affects the FNO only through validation data. The extension was not applied symmetrically at the time: the CNN’s point at the new rate was run only in the revision (Table I), and the FNO’s selected rate still sits at the edge of the extended grid, so the FNO may be slightly under-tuned as well. Each tuning cell is a single run on one seed.

What a full study would add. More PDEs and viscosities, out-of-distribution initial conditions, error beyond relative L_2 (e.g. spectra, shock location), more seeds and matched-convergence training, and the other neural operators discussed above.

VIII. Conclusion

In a five-seed CPU study on 1D Burgers, a reimplemented FNO is far more accurate than a DeepONet, an MLP and a CNN at 128 points with 400 training pairs and the same 100-epoch budget, and its advantage remains at 100 and 200 training pairs. The DeepONet and CNN baselines are under-fit at that budget, and the DeepONet result is weaker than published comparisons suggest, so the size of the gap should not be generalised. Applied zero-shot to 256 and 512 points, its error is unchanged, whereas the natively applied CNN degrades by factors of 2.55 and 3.27; the flat errors of MLP and DeepONet come from our subsampling rule and should not be read as invariance. These conclusions are limited to one equation,

band-limited data and a tuned-learning-rate-only baseline setup, and the resolution test is not a test of super-resolution beyond the training band.

References

- [1] N. Kovachki, Z. Li, B. Liu, K. Azizzadenesheli, K. Bhattacharya, A. Stuart, and A. Anandkumar, “Neural Operator: Learning Maps Between Function Spaces,” arXiv preprint arXiv:2108.08481, 2021. [Online]. Available: <https://arxiv.org/abs/2108.08481>
- [2] Z. Li, N. Kovachki, K. Azizzadenesheli, B. Liu, K. Bhattacharya, A. Stuart, and A. Anandkumar, “Fourier Neural Operator for Parametric Partial Differential Equations,” arXiv preprint arXiv:2010.08895, 2020. [Online]. Available: <https://arxiv.org/abs/2010.08895>
- [3] L. Lu, P. Jin, and G. E. Karniadakis, “DeepONet: Learning nonlinear operators for identifying differential equations based on the universal approximation theorem of operators,” arXiv preprint arXiv:1910.03193, 2019. [Online]. Available: <https://arxiv.org/abs/1910.03193>
- [4] L. Lu, X. Meng, S. Cai, Z. Mao, S. Goswami, Z. Zhang, and G. E. Karniadakis, “A comprehensive and fair comparison of two neural operators (with practical extensions) based on FAIR data,” arXiv preprint arXiv:2111.05512, 2021. [Online]. Available: <https://arxiv.org/abs/2111.05512>
- [5] M. Takamoto, T. Praditia, R. Leiteritz, D. MacKinlay, F. Alesiani, D. Pflüger, and M. Niepert, “PDEBENCH: An Extensive Benchmark for Scientific Machine Learning,” arXiv preprint arXiv:2210.07182, 2022. [Online]. Available: <https://arxiv.org/abs/2210.07182>
- [6] F. Bartolucci, E. de Bézenac, B. Raonić, R. Molinaro, S. Mishra, and R. Alaifari, “Representation Equivalent Neural Operators: a Framework for Alias-free Operator Learning,” arXiv preprint arXiv:2305.19913, 2023. [Online]. Available: <https://arxiv.org/abs/2305.19913>
- [7] K. Bhattacharya, B. Hosseini, N. B. Kovachki, and A. M. Stuart, “Model Reduction and Neural Networks for Parametric PDEs,” arXiv preprint arXiv:2005.03180, 2020. [Online]. Available: <https://arxiv.org/abs/2005.03180>
- [8] Z. Li, H. Zheng, N. Kovachki, D. Jin, H. Chen, B. Liu, K. Azizzadenesheli, and A. Anandkumar, “Physics-Informed Neural Operator for Learning Partial Differential Equations,” arXiv preprint arXiv:2111.03794, 2021. [Online]. Available: <https://arxiv.org/abs/2111.03794>
- [9] B. Raonić, R. Molinaro, T. D. Ryck, T. Rohner, F. Bartolucci, R. Alaifari, S. Mishra, and E. de Bézenac, “Convolutional Neural Operators for robust and accurate learning of PDEs,” arXiv preprint arXiv:2302.01178, 2023. [Online]. Available: <https://arxiv.org/abs/2302.01178>
- [10] A.-K. Kassam and L. N. Trefethen, “Fourth-Order Time-Stepping for Stiff PDEs,” SIAM Journal on Scientific Computing, 2005. [Online]. Available: <https://doi.org/10.1137/S1064827502410633>
- [11] D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” arXiv preprint arXiv:1412.6980, 2014. [Online]. Available: <https://arxiv.org/abs/1412.6980>